

Bug - 1 : The Query used to get data from database

File: TaskRepository.java

Cause: AND/OR precedence in the query. I saw the query involving multiple AND, OR operators without proper grouping / structuring.

Fix : Used Specification class which is used to create dynamic SQL queries by hibernate. Used its criteria builder to handle the request params.

Bug - 2 : Pagination

File: TaskController.java

Cause: The existing code fetched all records and sliced the few records that would fit in that page range.

Fix : Used Pageable & Page classes which are used to handle pagination while querying by hibernate. along with sorting. This also changed the response structure. So, the frontend is also changed according to the response.

Bug - 3 : Table stuck on "loading" after an error.

File: UseTasks.js

Cause: I ran the frontend when the backend is not running. I stuck in 'loading' screen while the error is shown in console.

Fix : So, If an error is caught in the catch block the state of loading is set to false & setting the state of error null for next mounting.

Bug - 4 : After change in filter, the page number is constant.

File: App.jsx.

Cause: Everytime the filter is changed, the state of the page is ~~same~~ same. It is showing page 3 while the filtered result has only 2 records.

Fix: Everytime the status & query changes, the state of page is set to '1' using handler functions.

Bug - 5 : Triggering api call for every key pressed.

File: App.jsx

Cause: If I type a query of length 1, it triggers 1 api calls and clearing each one triggers another 1 calls.

Fix: Creating a custom hook called useDebounce.js. which waits for 100ms after typing each key. When no key is typed for 100ms. Then it triggers an api call with query that has been typed so far.

Fix: Total pages calculation.

File: App.jsx

Cause: The App.jsx calculated total pages from total records found and page size.

Fix: As we used Page in backend, its default response contains all meta-data about pagination so we use that information here.